COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/24/2026	**DATE SUBMITTED:**	07/29/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 Linked Lists

1. PROCEDURE OUTPUT

What procedures did you perform?

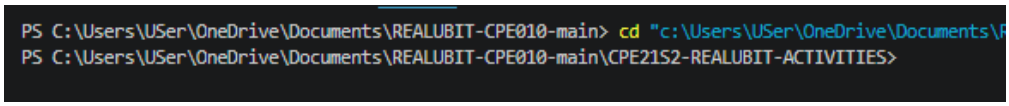
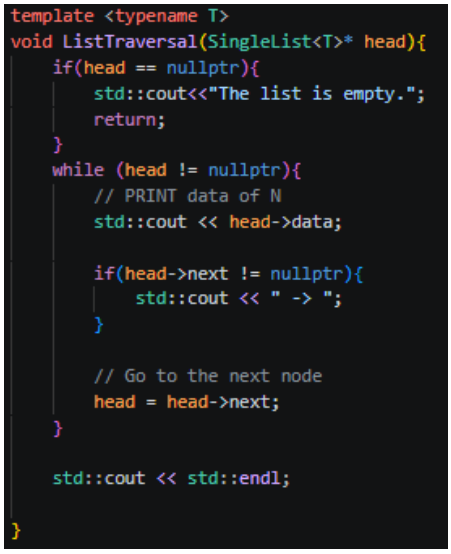
Screenshot	
Discussion	This is the output of the given code. As seen in the picture, there is nothing but the mark of the code being run. This is because, while we have created a linked list, we never really performed any operations that would actually show the linked list we made. Therefore, I think we can improve it more by actually adding a cout to show the implementation of the linked list in the terminal and the implementation of the header file to show the capabilities of a linked list.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	

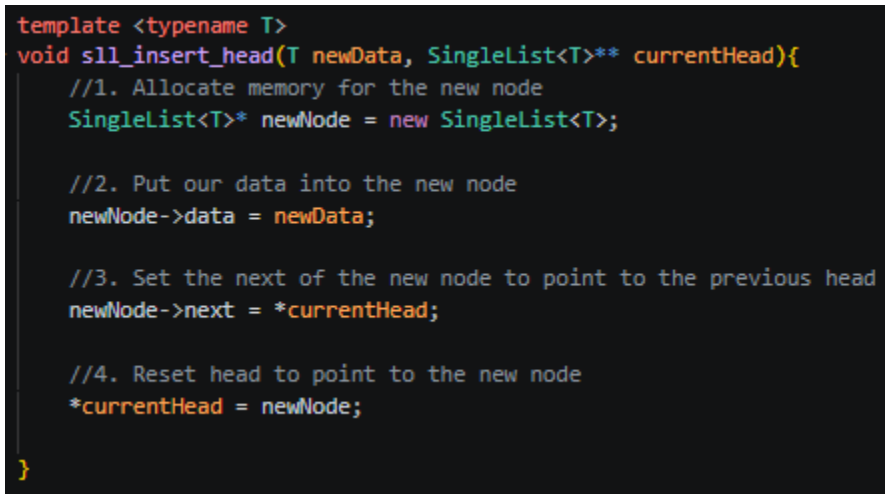
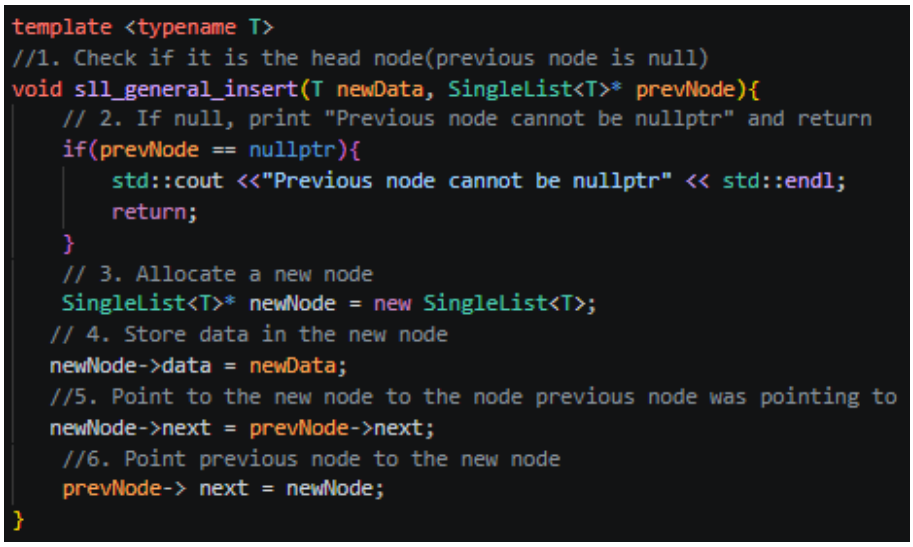
```
template <typename T>
void ListTraversal(SingleList<T>* head){
    if(head == nullptr){
        std::cout<<"The list is empty.";
        return;
    }
    while (head != nullptr){
        // PRINT data of N
        std::cout << head->data;

        if(head->next != nullptr){
            std::cout << " -> ";
        }

        // Go to the next node
        head = head->next;
    }

    std::cout << std::endl;

}
```

Insertion at Head	
Insertion at any part of the list	

Insertion at the end	
Deletion of a node	

```cpp
template <typename T>
void sll_insert_end(T newData, SingleList<T>** head){
    //1. Allocate new node
    SingleList<T>* newNode = new SingleList<T>;
    //2. Dereference to the head node
    SingleList<T>* currentNode = *head;
    //3. Store data in new node
    newNode->data = newData;
    //4. Point to null
    newNode-> next = nullptr;
    //5. Traverse the list until pointed to null
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }
    //6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

```cpp
template <typename T>
void sll_Delete_Node(T findData, SingleList<T>** head ){
    //if there is nothing to delete
    if(*head == nullptr) return;

    //
    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

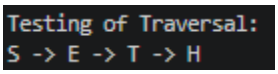
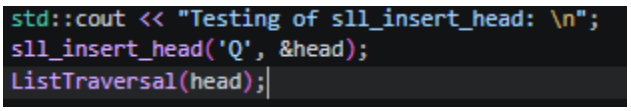
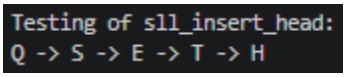
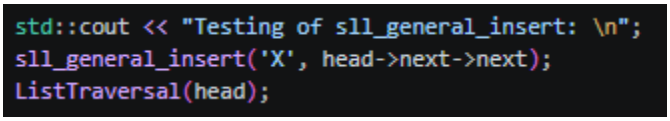
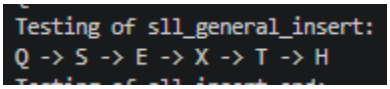
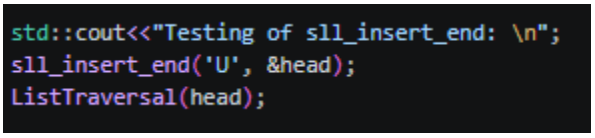
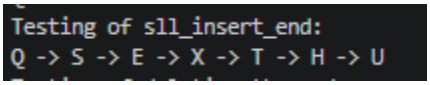
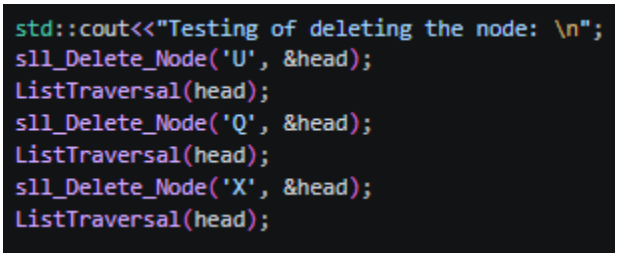
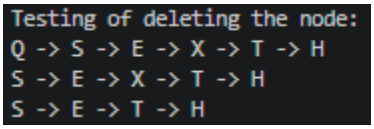
    // Searching the node: 1. the node does not exist + we found the data
    while(currNode != nullptr && currNode->data != findData){
        prevNode =currNode;
        currNode = prevNode->next;
    }
    //if data not found currNode == nullptr in the while loop:
    if(currNode == nullptr){
        std::cout<<"The data is not found \n"<<std::endl;
        return;
    }
    //if data was found
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }
    delete currNode;
}
```

Table 3-2. Code for the List Operations

a.	**Source Code**	

```cpp
std::cout << "Testing of Traversal: \n";
ListTraversal(head);
```

Figure 1. Code Snippet of Testing the function ListTraversal

	Console	 *Figure 2. Snippet of the Output in the console from ListTraversal*
b.	**Source Code**	 *Figure 3. Code Snippet of Testing the function sll_insert_head*
	Console	 *Figure 4. Snippet of the Output inside the console from sll_insert_head*
c.	**Source Code**	 *Figure 5. Code Snippet of the sll_general_insert on main.cpp*
	Console	 *Figure 6. Snippet of the Output inside the console from sll_general_insert*
d.	**Source Code**	 *Figure 7. Code Snippet of the function sll_insert_end to add U*
	Console	 *Figure 8. Snippet of the Output inside the console of adding U at the end*
e.	**Source Code**	 *Figure 9. Code Snippet of the function sll_Delete_Node*
	Console	

```
Testing of Traversal:
S -> E -> T -> H
```

```
std::cout << "Testing of sll_insert_head: \n";
sll_insert_head('Q', &head);
ListTraversal(head);
```

```
Testing of sll_insert_head:
Q -> S -> E -> T -> H
```

```
std::cout << "Testing of sll_general_insert: \n";
sll_general_insert('X', head->next->next);
ListTraversal(head);
```

```
Testing of sll_general_insert:
Q -> S -> E -> X -> T -> H
```

```
std::cout<<"Testing of sll_insert_end: \n";
sll_insert_end('U', &head);
ListTraversal(head);
```

```
Testing of sll_insert_end:
Q -> S -> E -> X -> T -> H -> U
```

```
std::cout<<"Testing of deleting the node: \n";
sll_Delete_Node('U', &head);
ListTraversal(head);
sll_Delete_Node('Q', &head);
ListTraversal(head);
sll_Delete_Node('X', &head);
ListTraversal(head);
```

```
Testing of deleting the node:
Q -> S -> E -> X -> T -> H
S -> E -> X -> T -> H
S -> E -> T -> H
```

		Figure 10. Snippet of the Output inside the console of deleting characters inside the linked lists

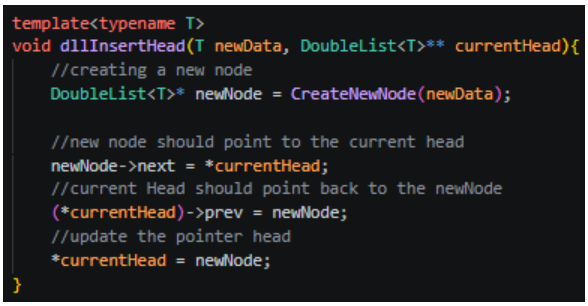
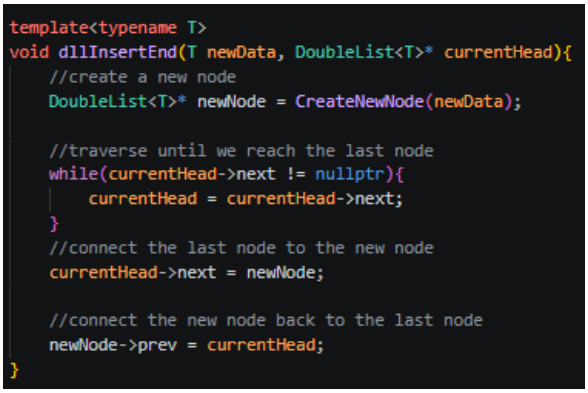
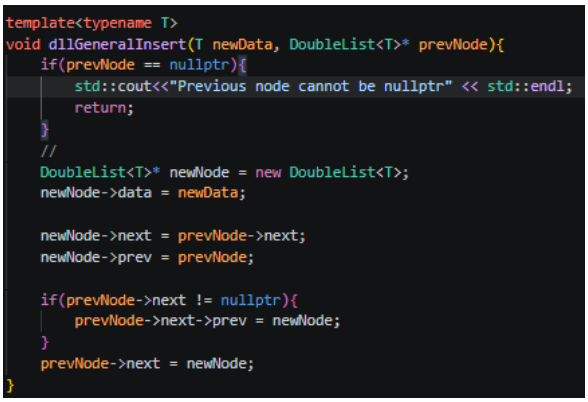
f.	**Source Code**	`std::cout<<"Deleting all nodes: \n";`
		`sll_Deleting_List(&head);`
		`ListTraversal(head);`
		Figure 11. Code Snippet of the function sll_deleting_list

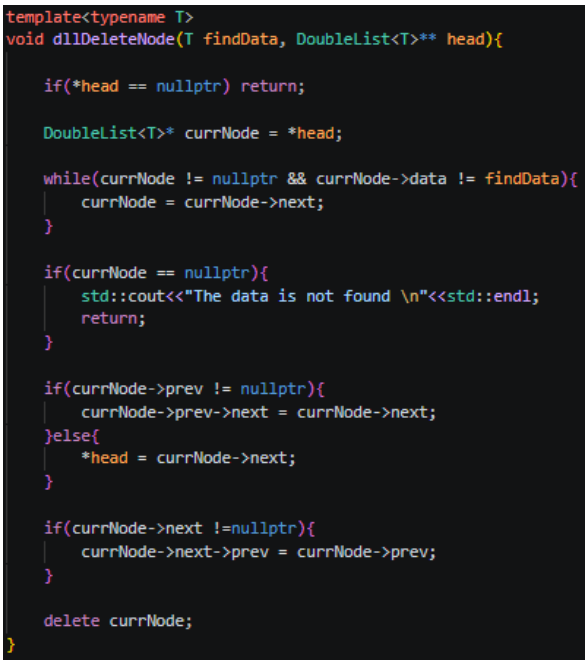
	Console	`Deleting all nodes:`
		`The list is empty.`
		Figure 12. Snippet of the Output inside the console for emptying the linked list

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis

`template <typename T>`
`void dllTraverse(DoubleList<T>* currentNode){`
`  // temp variable to store the last node visited`
`  DoubleList<T> *tail;`
`  //check if the list is empty`
`  if(currentNode == nullptr){`
`  std::cout<<"The List is empty"<<std::endl;`
`  return;`
`  }`
`  //move forward`
`  while(currentNode != nullptr){`
`  std::cout<< currentNode->data << " ";`
`  tail = currentNode;`
`  currentNode = currentNode->next;`
`  }`
`  //add a next line`
`  std::cout<<std::endl;`
`  //move backward`
`  std::cout<<"Backward: \n";`
`  while(tail != nullptr){`
`  std::cout<< tail->data <<" ";`
`  tail = tail->prev;`
`  }`
`  std::cout<<std::endl;`
`}`
	There is not a big difference from the singly linked list header file, but the most noticeable difference is that there is a temp variable so we can easily access the “tail” for ease of access, just like currentNode. The biggest difference that I would say is the addition of being able to move to the previous node just by simply having an arrow pointed to prev.

	In insert head, instead of using the syntax "newNode->next = *currentHead" on the second line, we use parentheses with the pointer currentHead to dereference it and move back to newNode.
	For insert end, the only difference is that we need to connect the last node to the new node and the new node to the last node, but apart from that we still use the while loop for traversing the list. Additionally, if looked upon closer, we only have a single pointer, whereas compared to the single linked list, we use 2. This is because a singly linked list does not have a prev pointer, so there is no way for it to walk backwards.
	In general insert, we mostly have the same syntax apart from the if loop where if prevNode->next does not equal to nullptr, it is simply a guard because unlike singly, doubly has something to update and needs to fix the prev pointer.

	Lastly, in delete node, singly manually tracks the prevNode during the search loop, but with doubly, since prevNode->prev reads directly and the pointer already exists on the node itself.

```
template<typename T>
void dllDeleteNode(T findData, DoubleList<T>** head){

    if(*head == nullptr) return;

    DoubleList<T>* currNode = *head;

    while(currNode != nullptr && currNode->data != findData){
        currNode = currNode->next;
    }

    if(currNode == nullptr){
        std::cout<<"The data is not found \n"<<std::endl;
        return;
    }

    if(currNode->prev != nullptr){
        currNode->prev->next = currNode->next;
    }else{
        *head = currNode->next;
    }

    if(currNode->next !=nullptr){
        currNode->next->prev = currNode->prev;
    }

    delete currNode;
}
```

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Main.cpp file (named HOA_ACT3_supp.cpp for convenience)

HOA_ACT3_supp.cpp | playlist_ll.h

hoa3p1 > HOA_ACT3_supp.cpp > main()

```cpp
#include <iostream>
#include "playlist_ll.h"

int main(){
    CircularList<std::string>* head = new CircularList<std::string>();
    CircularList<std::string>* second = new CircularList<std::string>();
    CircularList<std::string>* third = new CircularList<std::string>();
    CircularList<std::string>* songPlaying = head;

    head->data = "Song A";
    second->data = "Song B";
    third-> data = "Song C";

    head->next = second;
    second->next = third;
    third->next = head;

    head->prev = third;
    second->prev = head;
    third->prev = second;

    // Testing play_All
    std::cout<<"Playlist: "<< std::endl;
    play_All(head);

    //Testing insert_Head
    std::cout<<"New song added at head."<<std::endl;
    insert_Head(std::string("Song D"), &head);
    std::cout<<"Playlist: "<< std::endl;
    play_All(head);

    //Testing insert_End
    std::cout<<"New song added at end."<<std::endl;
    insert_End(std::string("Song E"), head);
    std::cout<<"Playlist: "<<std::endl;
    play_All(head);
```

Figure 1. Code Snippet of main.cpp with the functions: play_All, insert_Head, and insert_End

```
37
38      //Testing display_Current, next_Song and prev_Song
39      std::cout<<"Current Song Playing: ";
40      display_Current(songPlaying);
41      next_Song(&songPlaying);
42
43      std::cout<<"Current Song Playing: ";
44      display_Current(songPlaying);
45      prev_Song(&songPlaying);
46
47      std::cout<<"Current Song Playing: ";
48      display_Current(songPlaying);
49
50      destroy_List(&head);
51      play_All(head);
52      std::cout<<"Playlist destroyed."<<std::endl;
53
54      return 0;
55  }
56
```

Figure 2. Code Snippet of main.cpp with functions for display_Current, prev_song, next_song, and destroy_List

Header file

HOA_ACT3_supp.cpp | playlist_ll.h

hoa3p1 > playlist_ll.h > delete_Node<T>(T, CircularList<T>**, CircularList<T>**)

```cpp
#ifndef PLAYLIST_LL_H
#define PLAYLIST_LL_H
#include <iostream>
#include <string>

template<typename T>
class CircularList{
    public:
        T data;
        CircularList<T>*next = nullptr;
        CircularList<T>*prev = nullptr;
};

template<typename T>
CircularList<T>* CreateNewNode(T newData){

    CircularList<T> *newNode = new CircularList<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

template<typename T>
void play_All(CircularList<T>* head){
    if(head == nullptr){
        std::cout<<"The playlist is empty." << std::endl;
        return;
    }
    CircularList<T>* currentNode = head;
    do{
        std::cout << currentNode->data << " ";
        currentNode = currentNode->next;
    }while(currentNode != head);
    std::cout << std::endl;
}
```

Figure 3. Code Snippet of the class and functions play_All

```
37
38  template<typename T>
39  void insert_Head(T newData, CircularList<T>** currentHead){
40      CircularList<T>* newNode = CreateNewNode(newData);
41
42      if(*currentHead == nullptr){
43          newNode->next = newNode;
44          newNode->prev = newNode;
45          *currentHead = newNode;
46          // empty list into adding the first and only song
47          return;
48      }else{
49          newNode->next = *currentHead; // Song D's next -> Song A
50          newNode->prev = (*currentHead)->prev; // Song D's prev -> Song C
51          newNode->prev->next = newNode; // Song C's next -> Song D (was previously pointing to Song A)
52      (*currentHead)->prev = newNode; //Song A's prev -> Song D (was previously pointing to Song C)
53      *currentHead = newNode; // head label -> Song D
54
55      }
56  }
```

Figure 4. Code Snippet of insert_Head function

```
57
58  template<typename T>
59  void insert_End(T newData, CircularList<T>* currentHead){
60      CircularList<T>* newNode = CreateNewNode(newData);
61      //List is already circular so head ->prev is last node.
62      //No need for a traversal loop due to the shortcut of lastNode = currentHead->prev;
63      CircularList<T>* lastNode = currentHead->prev;
64      newNode->prev = lastNode;
65      newNode->next = currentHead;
66      lastNode->next = newNode;
67      currentHead->prev = newNode;
68  }
```

Figure 5. Code Snippet of insert_End function

```
69
70  template<typename T>
71  void next_Song(CircularList<T>** songPlaying){
72      *songPlaying = (*songPlaying)->next;
73      // move forward one song
74  }
75
76  template<typename T>
77  void prev_Song(CircularList<T>** songPlaying){
78      *songPlaying = (*songPlaying)->prev;
79      // move backward one song
80  }
81  // To easily show the state of playlist.
82  template<typename T>
83  void display_Current(CircularList<T>* songPlaying){
84      if(songPlaying == nullptr){
85          std::cout<<"There is currently no song playing." <<std::endl;
86          return;
87      }
88      std::cout<<songPlaying->data;
89      std::cout<<std::endl;
90  }
```

Figure 6. Code Snippet of functions: next_Song, prev_Song, display_Current

```cpp
template<typename T>
void delete_Node(T findData, CircularList<T>** head, CircularList<T>** songPlaying){
    if(*head == nullptr){
        std::cout << "The playlist is empty." <<std::endl;
        return;
    }
    // search the circle for a node matching findData
    CircularList<T>* currNode = *head;
    do{
        if(currNode->data == findData) break;
            currNode = currNode->next;
    }while(currNode != *head);
        // loop ends without break; just going back to head
    if(currNode->data != findData){
            std::cout << "Song not found in the playlist." <<std::endl;
            return;
    }
    // When currNode is pointing to itself
    if(currNode->next == currNode){
        *head = nullptr;
        *songPlaying = nullptr;
        delete currNode;
        return;
    }
    // If currNode is in the head, but other nodes still exist
    if(currNode == *head){
        currNode->prev->next = currNode->next;
        currNode->next->prev = currNode->prev;
        *head = currNode->next;
        if(*songPlaying == currNode){
            *songPlaying = currNode->next;

        }
        delete currNode;
        return;
    // If currNode is in the middle or a different node, head will stay the same
    }else{
        currNode->prev->next = currNode->next;
        currNode->next->prev = currNode->prev;
        if(*songPlaying == currNode){
            *songPlaying = currNode->next;
        }
        delete currNode;
    }
}
```

Figure 7. Code Snippet of the function, delete_Node and its multiple functions

```
137
138  template<typename T>
139  void destroy_List(CircularList<T>** head){
140      if(*head == nullptr);
141
142      CircularList<T>* currNode = *head;
143      do{
144          //save in a temp before deleting since currNode->next would be undefined after deletion.
145          CircularList<T>* temp = currNode;
146          currNode = currNode->next;
147          delete temp;
148
149      } while(currNode != *head);
150
151      *head = nullptr;
152  }
153
154  #endif
```

Figure 8. Code Snippet of the function, destroy_List

Output

```
Playlist:
Song A Song B Song C
New song added at head.
Playlist:
Song D Song A Song B Song C
New song added at end.
Playlist:
Song D Song A Song B Song C Song E
Current Song Playing: Song A
Current Song Playing: Song B
Current Song Playing: Song A
The playlist is empty.
Playlist destroyed.
```

Figure 9. Snippet of the Output inside the Terminal

Conclusion

This activity or hands-on activity was all about linked lists as a way of organising data. Instead of just relying on arrays, linked lists allow it to be dynamic, making it easier to traverse through and arrange the data. At first, just mindlessly copying the code for a singly linked list was overwhelming due to my lack of studying the topic beforehand.

We learned new keywords such as next and "prev", which had different uses of their own. The "next" was for chaining one node to another, while the "prev" was exclusive to doubly linked lists for it to traverse backwards through the linked list. Apart from that, we also learned how operations like insertion and deletion require carefully ordering the connections rather than just changing entire blocks of memory like an array.

Building the last two functions of doubly linked lists was slightly challenging; while I more or less understood the

concept, I always have a difficult time actually doing it in syntax. The difference between singly and doubly linked lists while making the procedure was different but, in a way, was also the same. While making both, it was very noticeable the difference in how the singly linked list used the prev pointer compared to the doubly.

For the doubly linked list, it had a direct assignment of the new node's prev, with a conditional check to update the following node's prev pointer. From the similar functions of the two, the most different was the deletion function. The singly version needed to track a prevNode variable throughout the search loop and needed to update it every iteration. Meanwhile, the doubly version only needed to use the syntax "currNode->prev" because it was easily accessible the moment it was found, turning what was a long loop of tracking into a simple lookup version.

The supplementary activity was the most difficult; while in a sense it is just the same, the logic was entirely different because we needed to make a circular linked list or remove nullptr from the linked list itself. The logic we learned was "if it's the end, then it is equal to nullptr", which made us think deeper about what we actually made.

The first was the traversal function (play_All). At first, I tried using the doubly linked list traversal in order to make the traversal function but failed because of the required condition, despite debugging it. This then escalated into making a do-while loop that went around from the starting node and then stopped once it wrapped back around since a while. Following that are the insertion operations; this was the second hardest to do because of the complexity. Since the list is circular, both the head and last node are and must always connect to each other, this meant that every insertion would require more pointers than what we were used to using the "next" and "prev". The doubly linked list provided a shortcut that would always point to the last node, "head->prev", albeit not directly; this proved to be valuable in operations like insert_end, essentially reducing the required logic and syntax to traverse the list.

The most difficult thing I found was the deletion; starting from the parameters themselves, it took me quite a considerable amount of time to add another parameter apart from just "findData" but "songPlaying" as well. Deleting the node itself is quite complex since it requires a relink from the space you deleted; therefore, it also led to needing multiple cases. One, deleting the only node in the list (requiring resetting the list to empty because it is only pointing to itself), two, deleting the head node (requiring reassigning the head pointer to the nearby nodes), and lastly, deleting any other node (needing to relink to the variables beside it). While not required, I added another function, which was the display_Current, for easy access to see what is happening to the linked list and the destroy_List, since in my logic, a playlist has to stop at one point, but deleting the actual playlist would be wrong since it might lead to an error, so I reassigned it into a temporary variable then deleted it afterwards.

Overall, the activity was highly difficult, and if I could say so, it was rightly difficult. While my skills are not the best, I make do with some help from different resources and available repositories as a reference. This only proved my lack of critical thinking and was further proved by blanking out on how I should actually start the syntax itself without referencing my previous works or the works of others. While it is not entirely wrong to reference, it would still be better to be able to make it on your own, which would be a good sign of progress in my required skills.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly AI and Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammarly correction, Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly AI was primarily used in fixing grammar issues such as a better choice of words from what I'm implying, comma's missed periods and etc. Meanwhile, for Claude AI, due to its nature as an LLM, I strictly told it to refrain from giving any line of code. Compared to Gemini or Google AI, which was inconsistent and occasionally gave solutions of code itself rather than just the concept of what I ask for. Therefore, with this usage, it does not explicitly go beyond what was allowed in the usage of AI in just figuring out the error and asking if the solution had no errors like how a code visualizer would act.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.